

Python Asyncio and Pydantic: Practical Foundations

This document covers the most common Python concepts you need to understand **asynchronous programming with `asyncio`** and **data validation with Pydantic**.

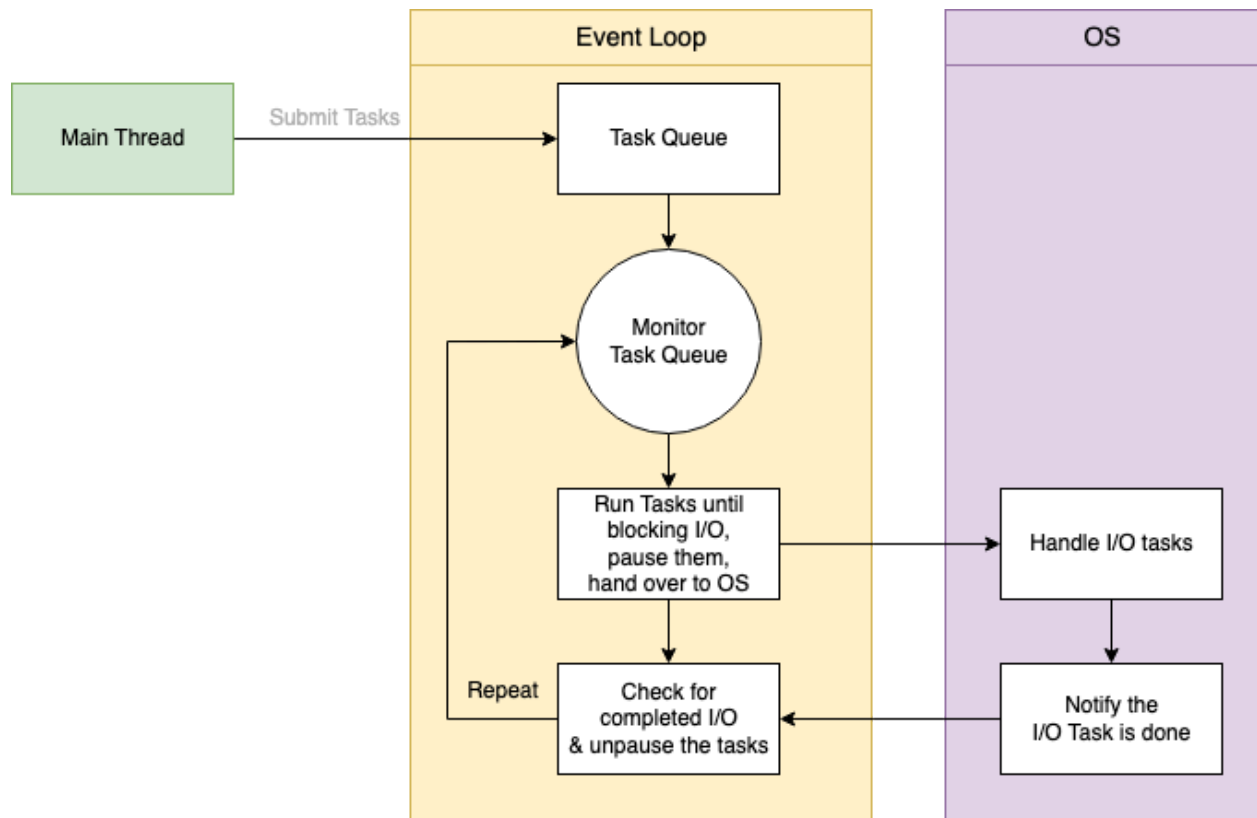
You do not need to memorize every API. The goal is to understand the mental models and common patterns so that you can confidently read and write Python code used in modern AI applications.

Event Loop

To achieve a single-threaded concurrency model, the `asyncio` package uses a construct called an event loop.

How it works.

- First, the main thread submits tasks to a task queue.
- Second, the event loop constantly monitors the task queue and runs the task until it counters I/O tasks. In this case, the event loop pauses the task and hands it over to the OS.
- Third, check for the completed IO tasks. If the task is completed, the OS will notify the program. The event loop then runs the unpaused tasks.



Part 1: Asyncio and Async Python

1. Why Async Programming?

Many applications spend a significant amount of time **waiting**.

For example:

- Calling an LLM API
- Calling another REST API
- Querying a database
- Reading from a network
- Waiting for a file or service
- Streaming data

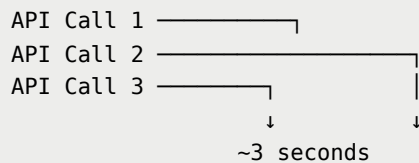
Consider three API calls:

```
API Call 1 → 2 seconds  
API Call 2 → 3 seconds  
API Call 3 → 1 second
```

If we execute them sequentially:

```
2 + 3 + 1 = 6 seconds
```

If the operations are independent, we can perform them concurrently:



Python provides the `asyncio` module for this type of **I/O-bound concurrency**.

2. `async def`

An asynchronous function is defined using `async def`.

```
async def say_hello():  
    print("Hello")
```

Calling it does not immediately execute the function in the normal way.

```
result = say_hello()
```

This creates a **coroutine object**.

To actually run the coroutine, it needs to be executed by an event loop.

3. `await`

Inside an async function, we can use `await`.

```
import asyncio

async def say_hello():
    print("Hello")
    await asyncio.sleep(1)
    print("World")
```

`await` means:

Pause this coroutine until the awaited operation is ready, while allowing the event loop to run other work.

This is especially useful for I/O operations.

```
await api_call()
await database_query()
await asyncio.sleep(1)
```

The important distinction is:

```
await ≠ blocking
```

An asynchronous `await` allows other async work to make progress while this coroutine is waiting.

4. Running an Async Program

For a standalone Python program, the simplest way to start the event loop is:

```
asyncio.run(main())
```

Example:

```
import asyncio

async def main():
    print("Hello from async Python")

asyncio.run(main())
```

A common pattern is:

```
async def main():
    # async application logic
    pass

if __name__ == "__main__":
    asyncio.run(main())
```

5. `asyncio.sleep()`

`asyncio.sleep()` is useful for understanding asynchronous execution.

```
import asyncio

async def task():
    print("Started")
    await asyncio.sleep(2)
    print("Finished")

asyncio.run(task())
```

Unlike:

```
import time

time.sleep(2)
```

`asyncio.sleep()` does not block the event loop.

This distinction becomes important when many operations are running concurrently.

6. Concurrency with `asyncio.gather()`

Suppose we have three independent operations:

```
async def task1():
    await asyncio.sleep(2)
    return "Task 1 completed"

async def task2():
    await asyncio.sleep(3)
    return "Task 2 completed"

async def task3():
    await asyncio.sleep(1)
    return "Task 3 completed"
```

We can execute them concurrently:

```
results = await asyncio.gather(
    task1(),
    task2(),
    task3()
)
```

```
print(results)
```

The result will be:

```
[
    "Task 1 completed",
    "Task 2 completed",
    "Task 3 completed"
]
```

The total waiting time is approximately the longest individual task:

```
max(2, 3, 1) = 3 seconds
```

rather than:

```
2 + 3 + 1 = 6 seconds
```

When should you use `gather()` ?

Use it when:

- Operations are independent
- They are I/O-bound
- You want them to execute concurrently

For example:

```
user = await get_user()
posts = await get_posts()
todos = await get_todos()
```

can potentially become:

```
user, posts, todos = await asyncio.gather(
    get_user(),
    get_posts(),
    get_todos()
)
```

7. Async HTTP Requests

A common use of asyncio in AI applications is making HTTP requests.

A synchronous HTTP library such as `requests` blocks while waiting for a response.

For asynchronous HTTP requests, libraries such as `httpx` provide an async client.

```
import httpx

async def get_data():
    async with httpx.AsyncClient() as client:
        response = await client.get(
            "https://jsonplaceholder.typicode.com/posts/1"
        )

    return response.json()
```

Notice three important concepts:

```
async def get_data():
```

The function is asynchronous.

```
async with httpx.AsyncClient()
```

The HTTP client is an asynchronous resource.


```
await client.get(...)
```

The network operation is asynchronous.

8. `async with`

Some objects need to be opened and closed properly.

For example:

```
async with httpx.AsyncClient() as client:
    response = await client.get(url)
```

This is similar in concept to:

```
with open("file.txt") as file:
    ...
```

The difference is that `async with` is used with an **asynchronous context manager**.

You will commonly encounter it with:

- Async HTTP clients
 - Async database connections
 - Async sessions
 - Other async resources
-

9. `async for`

Some asynchronous operations produce values over time.

For example, an LLM may stream its response:

```
Hello
Hello, how
```

```
Hello, how are  
Hello, how are you?
```

An asynchronous stream can be consumed using:

```
async for chunk in stream:  
    print(chunk)
```

This is the asynchronous equivalent of iterating over a normal iterable:

```
for item in items:  
    print(item)
```

You will commonly encounter `async for` when working with:

- LLM streaming
- Async generators
- Streaming APIs
- Async event streams

10. Async Generators

A function containing both `async def` and `yield` is an async generator.

```
async def generate_numbers():  
    for i in range(3):  
        await asyncio.sleep(1)  
        yield i
```

It can be consumed using:

```
async for number in generate_numbers():  
    print(number)
```

Output:

```
0  
1  
2
```

This pattern is particularly relevant to AI applications because streaming responses are often represented as async iterators.

11. Async LLM Calls

Modern AI SDKs commonly provide asynchronous APIs.

For example, an SDK may expose:

```
response = await client.chat.completions.create(  
    model="some-model",  
    messages=[  
        {"role": "user", "content": "Explain RAG"}  
    ]  
)
```

The important part is not the specific SDK.

The pattern is:

```
async function  
  ↓  
await  
  ↓  
network request  
  ↓  
response
```

Later, AI frameworks may expose similar APIs with names such as:

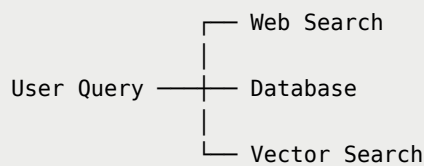
```
await model.ainvoke(...)
```

or:

```
async for chunk in model.astream(...):  
    ...
```

12. Async Concurrency in AI Applications

Imagine an AI agent needs information from three independent sources:



These operations may be performed concurrently:

```
web_result, db_result, documents = await asyncio.gather(  
    web_search(query),  
    database_search(query),  
    vector_search(query)  
)
```

This can reduce latency because the application does not need to wait for each independent operation sequentially.

This pattern appears frequently in:

- RAG systems
- AI agents
- Tool calling
- Multi-agent systems
- API orchestration
- Data pipelines

13. Async vs Parallelism

These concepts are related but not the same.

Async concurrency

Multiple I/O operations make progress while waiting for each other.

```
Request A — wait ————— response
Request B — wait ——— response
Request C — wait ————— response
```

Parallelism

Multiple CPU operations actually execute simultaneously, typically using multiple CPU cores or processes.

Asyncio is primarily designed for **I/O-bound concurrency**.

For example:

```
Good use of asyncio:
- HTTP requests
- Database calls
- LLM calls
- Network operations
```

It is not a magic solution for CPU-heavy work such as:

```
- Large numerical calculations
- CPU-intensive image processing
- Heavy data processing
```

14. The Most Important Asyncio Concepts

You should be comfortable reading code containing:

```
async def
await
asyncio.run()
asyncio.gather()
async with
async for
```

A useful mental model is:

```
async def
  ↓
Defines an async function

await
  ↓
Pause this coroutine while waiting for async work

asyncio
  ↓
Provides the machinery for running and coordinating async code

gather()
  ↓
Run independent async operations concurrently

async for
  ↓
Consume asynchronous streams

async with
  ↓
Manage asynchronous resources
```

You do not need to memorize the entire `asyncio` library before working with AI systems.

Part 2: Pydantic

15. Why Pydantic?

AI applications deal with a lot of structured data.

For example, an LLM might return:

```
{
  "name": "John",
  "age": 28,
  "email": "john@example.com"
}
```

We want to make sure that this data has the structure our application expects.

Pydantic allows us to define the expected structure using Python classes and type annotations.

16. Installing Pydantic

```
%pip install pydantic
```

Import it:

```
from pydantic import BaseModel
```

17. Creating a Pydantic Model

A Pydantic model is a Python class that extends `BaseModel`.

```
from pydantic import BaseModel
```

```
class User(BaseModel):
    name: str
    age: int
    email: str
```

Now we can create a user:

```
user = User(  
    name="John",  
    age=28,  
    email="john@example.com"  
)
```

Access fields like a normal Python object:

```
print(user.name)  
print(user.age)  
print(user.email)
```

18. Type Validation

Pydantic uses the type annotations to validate input.

```
class User(BaseModel):  
    name: str  
    age: int
```

This is valid:

```
user = User(  
    name="John",  
    age=28  
)
```

But invalid data can result in a validation error:

```
user = User(  
    name="John",
```



```
        age="hello"  
    )
```

Pydantic raises a `ValidationError`.

This is one of the major benefits of Pydantic:

Define the structure once and let Pydantic validate incoming data against it.

19. Required vs Optional Fields

By default, fields without a default value are required.

```
class User(BaseModel):  
    name: str  
    age: int
```

Both fields are required.

You can provide a default:

```
class User(BaseModel):  
    name: str  
    age: int = 18
```

Now `age` is optional during construction:

```
user = User(name="John")
```

The resulting object has:

```
user.age  
# 18
```

For a field that may explicitly contain `None`, use `Optional` or the modern union syntax:

```
class User(BaseModel):  
    name: str  
    nickname: str | None = None
```

20. Nested Pydantic Models

Real-world data is often nested.

For example:

```
{  
    "name": "John",  
    "address": {  
        "city": "Delhi",  
        "country": "India"  
    }  
}
```

We can represent this using nested models:

```
class Address(BaseModel):  
    city: str  
    country: str  
  
class User(BaseModel):  
    name: str  
    address: Address
```

Now:

```
user = User(  
    name="John",  
    address={
```

```
        "city": "Delhi",
        "country": "India"
    }
)
```

Pydantic converts the nested dictionary into an `Address` model.

```
print(user.address.city)
```

21. Lists of Objects

Pydantic also works with collections.

```
class User(BaseModel):
    name: str
    age: int

class Team(BaseModel):
    name: str
    members: list[User]
```

We can provide:

```
team = Team(
    name="AI Team",
    members=[
        {"name": "John", "age": 28},
        {"name": "Sarah", "age": 30}
    ]
)
```

Pydantic validates each member against the `User` model.

22. Dictionaries and Other Common Types

Pydantic works naturally with Python type hints.

For example:

```
class Product(BaseModel):  
    name: str  
    price: float  
    tags: list[str]  
    metadata: dict[str, str]
```

This tells Pydantic:

```
name      → string  
price     → floating-point number  
tags      → list of strings  
metadata  → dictionary with string keys and values
```

Common types you will use include:

```
str  
int  
float  
bool  
list  
dict  
set  
tuple
```

23. Pydantic and JSON

AI applications frequently receive JSON from APIs.

Suppose we receive:

```
data = {  
    "name": "John",  
    "age": 28,  
    "email": "john@example.com"  
}
```

We can validate it:

```
user = User.model_validate(data)
```

We can convert a Pydantic model back to a dictionary:

```
data = user.model_dump()
```

And to JSON:

```
json_data = user.model_dump_json()
```

These three operations are worth remembering:

```
Python dictionary  
  ↓  
model_validate()  
  ↓  
Pydantic object  
  ↓  
model_dump()  
  ↓  
Python dictionary
```

24. Validating JSON Strings

If you have JSON as a string:

```
json_data = """  
{
```

```
"name": "John",  
"age": 28,  
"email": "john@example.com"  
}  
"""
```

You can use:

```
user = User.model_validate_json(json_data)
```

This is useful when data arrives as JSON from an external system.

25. Field Constraints

Pydantic can validate more than just basic types.

For example:

```
from pydantic import BaseModel, Field  
  
class User(BaseModel):  
    name: str = Field(min_length=2)  
    age: int = Field(ge=18)
```

Now:

```
name → must contain at least 2 characters  
age → must be greater than or equal to 18
```

You can also define constraints such as:

```
Field(  
    min_length=1,
```

```
        max_length=100
    )
```

or:

```
Field(
    ge=0,
    le=100
)
```

Use constraints when the application has a meaningful business rule to enforce.

26. Default Values

You can provide defaults directly:

```
class Config(BaseModel):
    temperature: float = 0.7
    max_tokens: int = 1000
```

Then:

```
config = Config()
```

will produce:

```
config.temperature
# 0.7

config.max_tokens
# 1000
```

27. Pydantic Enum

For fields that should contain one of a fixed set of values, Python enums can be useful.

```
from enum import Enum
from pydantic import BaseModel

class Status(str, Enum):
    ACTIVE = "active"
    INACTIVE = "inactive"

class User(BaseModel):
    name: str
    status: Status
```

Now the status is constrained to known values.

```
user = User(
    name="John",
    status="active"
)
```

This is useful for:

- Status fields
- Categories
- User roles
- Agent states
- Fixed configuration options

28. Pydantic Validation Errors

When validation fails, Pydantic raises a `ValidationError`.


```

from pydantic import ValidationError

try:
    user = User(
        name="John",
        age="invalid"
    )
except ValidationError as e:
    print(e)

```

The error contains structured information about:

- Which field failed
- What type/value was expected
- What input was received

You can inspect the errors programmatically:

```

print(e.errors())

```

29. Pydantic Models for API Responses

Suppose an external API returns:

```

{
    "id": 1,
    "title": "Hello",
    "completed": true
}

```

We can define:

```

class Todo(BaseModel):
    id: int

```

```
title: str
completed: bool
```

Then:

```
todo = Todo.model_validate(response_data)
```

Now the rest of the application works with a predictable Python object:

```
print(todo.id)
print(todo.title)
print(todo.completed)
```

This is especially useful when integrating multiple external APIs.

30. Pydantic in AI Applications

Pydantic becomes particularly useful when working with LLMs.

An LLM might be asked:

| Extract the user's information from this message.

Instead of accepting arbitrary text, we can define the expected structure:

```
class UserInfo(BaseModel):
    name: str
    age: int
    city: str
```

The desired output becomes:

```
{
  "name": "John",
  "age": 28,
```

```
"city": "Delhi"  
}
```

The application can then validate the result against the Pydantic model.

This idea is central to **structured outputs** in modern LLM applications.

31. Pydantic Models for Tools and Agents

Agents often need structured input.

For example, a weather tool might require:

```
class WeatherRequest(BaseModel):  
    city: str  
    country: str
```

The agent can provide:

```
{  
    "city": "Delhi",  
    "country": "India"  
}
```

The application can validate the tool input before executing it.

The same concept can be used for:

- Tool arguments
 - Agent state
 - Configuration
 - API requests
 - API responses
 - Structured LLM output
-

32. Pydantic vs Type Hints

It is important to understand the difference.

A type hint:

```
def get_user(age: int):  
    ...
```

communicates that `age` should be an integer.

Python itself generally does not enforce this annotation at runtime.

Pydantic performs runtime validation:

```
class User(BaseModel):  
    age: int
```

When constructing the model, Pydantic checks the supplied data.

So:

```
Type hints  
  ↓  
Communicate expected types  
  
Pydantic  
  ↓  
Validates structured data at runtime
```

They work together rather than replacing each other.

33. Pydantic vs Dataclasses

Python also has `dataclasses`:

```
from dataclasses import dataclass
```

```
@dataclass
class User:
    name: str
    age: int
```

Dataclasses are useful for representing Python objects.

Pydantic models provide additional functionality around:

- Runtime validation
- Parsing external data
- JSON serialization
- Data constraints
- Structured input/output

For AI applications, APIs, and external data, Pydantic is particularly useful.

34. A Small Real-World Example

Imagine an AI application receives a request:

```
request_data = {
    "question": "What is RAG?",
    "max_results": 5
}
```

Define the expected structure:

```
class SearchRequest(BaseModel):
    question: str
    max_results: int = 5
```

Validate it:

```
request = SearchRequest.model_validate(request_data)
```

Now the application can safely use:

```
print(request.question)
print(request.max_results)
```

The flow becomes:

```
External Data
  ↓
Pydantic Validation
  ↓
Structured Python Object
  ↓
Application Logic
  ↓
LLM / Database / API
```

This pattern appears everywhere in production AI systems.

35. Asyncio + Pydantic Together

These two technologies solve different problems.

Asyncio

Deals with:

```
How do we efficiently wait for and coordinate I/O operations?
```

Pydantic

Deals with:

```
What structure and types should our data have?
```

They can be used together.

For example:

```

import httpx
from pydantic import BaseModel

class Todo(BaseModel):
    id: int
    title: str
    completed: bool

async def get_todo(todo_id: int) -> Todo:
    async with httpx.AsyncClient() as client:
        response = await client.get(
            f"https://jsonplaceholder.typicode.com/todos/{todo_id}"
        )

        response.raise_for_status()

    return Todo.model_validate(response.json())

```

Here:

```

asyncio
  ↓
Makes the HTTP request asynchronously

httpx
  ↓
Communicates with the API

Pydantic
  ↓
Validates the API response

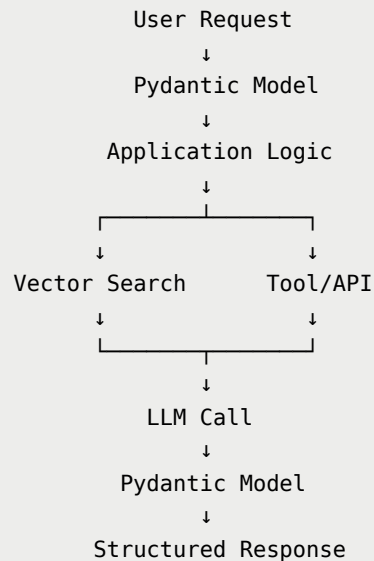
```

These are separate responsibilities that work well together.

36. Asyncio + Pydantic + LLMs

This combination appears frequently in AI engineering.

A simplified architecture might look like:



And the independent operations can be performed concurrently:

```
documents, tool_result = await asyncio.gather(
    retrieve_documents(query),
    call_tool(query)
)
```

The result can then be validated using Pydantic.

This combination is one of the foundations for building reliable AI applications.

37. What You Should Remember

You do not need to memorize every method in `asyncio` or Pydantic.

Focus on these core concepts.

Asyncio

```
async def
await
asyncio.run()
asyncio.gather()
async with
async for
```

Understand:

- What an async function is
- What `await` does
- Why async is useful for I/O
- How `gather()` enables concurrency
- How async HTTP requests work
- How async streaming works

Pydantic

```
BaseModel
Field
model_validate()
model_validate_json()
model_dump()
model_dump_json()
ValidationError
```

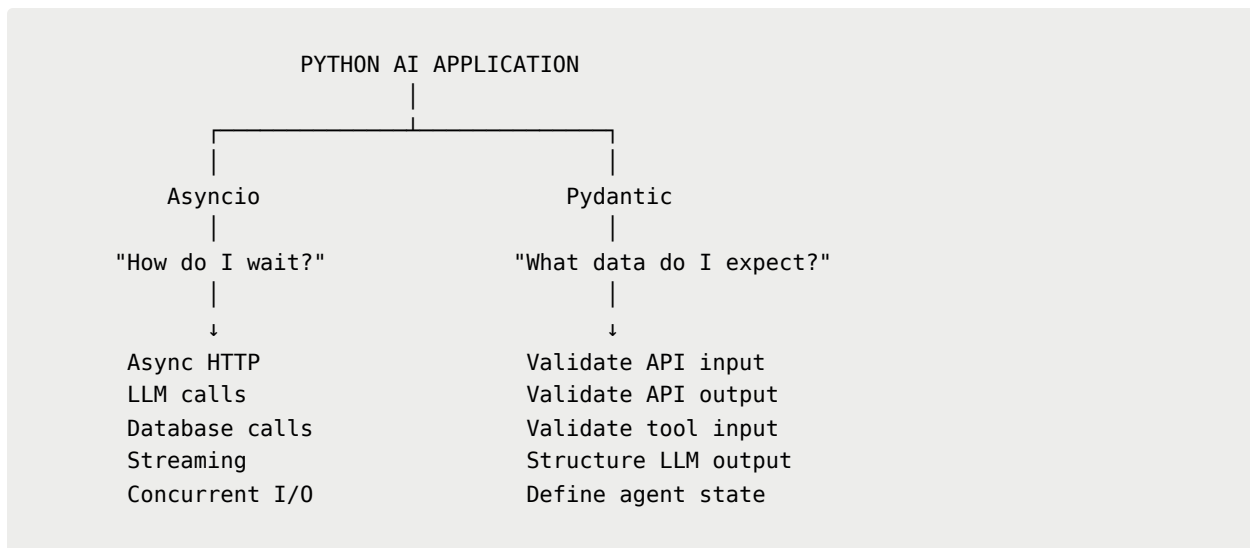
Understand:

- How to define a model
- Required and optional fields
- Nested models

- Lists and dictionaries
- Validation
- Default values
- Field constraints
- JSON parsing and serialization
- Structured data in LLM applications

38. Final Mental Model

Think of the two technologies this way:



The goal is not to become an expert in either library immediately.

The goal is to reach the point where, when you see code such as:

```
result = await model.ainvoke(request)
```

or:

```
async for chunk in model.astream(request):
    ...
```

or:

```
request = SearchRequest.model_validate(data)
```

you understand **what is happening, why it is being used, and how the pieces fit together.**